

1 TypeSpec Syntax Specification

Annotations	A	::=	$\overline{\text{@spec } E}$
Expressions	E	::=	$x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T	::=	$\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \ll \mid \ll :: n \gg \mid \ll :: *n \gg \mid \ll :: n, _ :: *n \gg$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F	::=	$\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\overline{\text{@spec } E}$ and $\overline{\alpha : \bar{T}}$, where it denotes a non-empty sequence.)

Figure 1: TypeSpec

1.1 TypeSpec Built-in Types and Syntactic Sugar

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpec Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k:T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpec Syntactic Sugars

2 Elixir Types Syntax Specification

Annotations	A ::= \$S \$S when $\overline{\alpha:T}$	
Signatures	S ::= $\overline{T} \rightarrow T$ S and S	
Types	T ::= α K term() none() dynamic() T or T T and T not T n..n infty..n n..infty { $\overline{T}$ } { $\overline{T}, \dots$ } %{ $\overline{F}$ } %{ $\dots, \overline{F}$ } ($\overline{T} \rightarrow T$) empty_list() non_empty_list(T, T)	type variables basic types type connectives integers (internal representation) type constructors
Key types	K ::= atom() :k pid() port() reference() float() integer() binary() tuple() open_map() fun() list()	
Fields	F ::= :k=>T K=>if_set(T)	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from { $\overline{T}, \dots$ }, %{ $\dots, \overline{F}$ }, and $\overline{\alpha:T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 2: Elixir Types Syntax

2.1 Elixir Type Built-in Types and Syntactic Sugar

Built-in types	Defined as
boolean()	:true or :false
non_empty_list(T)	non_empty_list(T, empty_list())
list(T, T')	empty_list() or non_empty_list(T, T')
list(T)	empty_list() or non_empty_list(T, T')
list()	empty_list() or non_empty_list(term(), empty_list())

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
$\{\dots\}$	<code>tuple()</code>
$\%\{\dots\}$	<code>open_map()</code>
$k:T$	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 4: Elixir Type Syntactic Sugars

3 Translation from TypeSpec to Elixir Types

$\llbracket \cdot \rrbracket$: TypeSpec $\rightarrow$ Elixir type

Figure 3: Semantic of Translation

3.1 Translation of TypeSpec Specification into Elixir Annotations

$\llbracket @spec\ E \rrbracket = \$ \llbracket E \rrbracket$

$\llbracket @spec\ \overline{E}, @spec\ E' \rrbracket = \$ \llbracket \overline{E}, E' \rrbracket$

$\llbracket x(\overline{T_1}) :: T_2 \rrbracket = \llbracket \overline{T_1} \rrbracket \rightarrow \llbracket T_2 \rrbracket$

$\llbracket x(\overline{T_1}) :: T_2 \text{ when } \overline{\alpha} : \overline{T_3} \rrbracket = \llbracket \overline{T_1} \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket$

$\llbracket \overline{E}, x(\overline{T_1}') :: T_2' \rrbracket = \begin{cases} S \text{ and } \llbracket \overline{T_1}' \rrbracket \rightarrow \llbracket T_2' \rrbracket \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket & \text{if } \llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket \\ \llbracket \overline{E} \rrbracket \text{ and } \llbracket \overline{T_1}' \rrbracket \rightarrow \llbracket T_2' \rrbracket & \text{otherwise} \end{cases}$

$\llbracket \overline{E}, x(\overline{T_1}') :: T_2' \text{ when } \overline{\alpha'} : \overline{T_3'} \rrbracket = \begin{cases} S \text{ and } \llbracket \overline{T_1}' \rrbracket \rightarrow \llbracket T_2' \rrbracket \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket, \overline{\alpha'} : \llbracket \overline{T_3'} \rrbracket & \text{if } \llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket \\ & \text{and } \overline{\alpha} \cap \overline{\alpha'} = \emptyset \\ S \text{ and } \llbracket \overline{T_1}' \rrbracket \rightarrow \llbracket T_2' \rrbracket[\alpha' := \beta'] \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket, \overline{\beta'} : \llbracket \overline{T_3'} \rrbracket & \text{if } \llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha} : \llbracket \overline{T_3} \rrbracket \\ & \text{and } \overline{\alpha} \cap \overline{\alpha'} \neq \emptyset \\ \llbracket \overline{E} \rrbracket \text{ and } \llbracket \overline{T_1}' \rrbracket \rightarrow \llbracket T_2' \rrbracket \text{ when } \overline{\alpha'} : \llbracket \overline{T_3'} \rrbracket & \text{otherwise} \end{cases}$

Figure 4: Translation into Annotations

3.2 Translation of TypeSpec into Elixir Types

```

[[α]] = α
[[T1 | T2]] = [[T1]] or [[T2]]
[[any()]] = term()
[[none()]] = none()
[[atom()]] = atom()
[[ :k ]] = :k
[[pid()]] = pid()
[[port()]] = port()
[[reference()]] = reference()
[[float()]] = float()
[[integer()]] = integer()
[[neg_integer]] = infy..-1
[[non_neg_integer]] = 0..infy
[[pos_integer]] = 1..infy
[[tuple()]] = tuple()
[[{T̄}]] = {[T̄]}
[[%{F1, ..., Fn}]] = map(⊕i=1n[[Fi]])
[[[]]] = empty_list()
[[nonempty_maybe_improper_list(T1, T2)] = non_empty_list([[T1]], [[T2]])
[[«»]] = undefined
[[«_::n»]] = { binary()    if n mod 8 = 0
               undefined otherwise
[[«_::_*n»]] = { binary()    if n mod 8 = 0
               undefined otherwise
[[«_::n, _::_*n»]] = { binary()    if n mod 8 = 0
                      undefined otherwise
[[ (T̄ -> T) ]] = ([[T̄]] -> [[T]])
[[ (... -> T) ]] = { fun()      if [[T]] ≈ term()
                   undefined otherwise
[[n]] = n..n
[[n..n']] = n..n'

```

Figure 5: Translation in Types

3.3 Translation of Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or `if_set(T)`. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by `Empty` the lists that maps all the non-singleton keytypes to `not_set()`.

Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $K \Rightarrow \tau$ of L_2 such that $K \notin L_1$. Formally:

$$\begin{aligned}
L_1 \oplus [] &= L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } K \in L_1 \\
L_1 \oplus [K \Rightarrow \tau, L_2] &= [K \Rightarrow \tau, L_1] \oplus L_2 && \text{if } K \notin L_1
\end{aligned}$$

Figure 6: Merge in Field Types

We then define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible keytypes), then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty})$.

All it remains to do is to define the translation of field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $[\bigvee_{i=1}^n K_i \Rightarrow \tau]$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 7.:

$$\begin{aligned}
\llbracket \text{required}(T) \Rightarrow T' \rrbracket &= \begin{cases} [:k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx :k \\ [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases} \\
\llbracket \text{optional}(T) \Rightarrow T' \rrbracket &= \begin{cases} [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

where all K_i 's in I are pairwise distinct.

Figure 7: Translation in Field Types

4 Total Approximating Translation of TypeSpec into Elixir Types

The translation defined so far is partial since some TypeSpec types do not have a corresponding Elixir type. This is not a weakness of the Elixir type system, but rather a consequence of the design choices made for Elixir types, to keep them as simple as possible while still being useful. However, it is possible to define a total translation from TypeSpec types to Elixir types that approximates the original TypeSpec type. This total translation can be useful in some contexts, for example when we want to ensure that every TypeSpec type can be represented in Elixir types, even if approximately.

More precisely, the previous translation is undefined in the following cases:

- Bitstrings with size specifications that are not a multiple of 8 bits (e.g., `<<_:3>>`).
- The type `... -> T` when the translation of T is not equivalent to `any()` (explain why this type is not interesting).
- (`n..n`)
- $\llbracket \text{required}(T) \Rightarrow T' \rrbracket$ and $\llbracket \text{optional}(T) \Rightarrow T' \rrbracket$ are undefined when $\llbracket T \rrbracket$ is not equivalent to a disjunction of pairwise distinct key types.

In the rest of this section we give a conservative extension of the translation defined so far that is total, i.e., it is defined for every TypeSpec type. The extension consists in defining approximations for the cases where the previous translation is undefined.

4.1 Approximation of Bitstring Types

Elixir Types has no plan to support bitstring types that is finer types than `binary()`, hence, it is not possible to directly translate any bitstring types. However, most of the bitstring representatives could be trivially translated to `binary()`. According to the Erlang documentation, bitstrings are a later generalization of binaries, so many texts and much information about binaries apply just as well for bitstrings. In other words, the purpose of bitstrings is to construct binaries in more precise manner. Elixir Types provides type segments as Figure 8 that specify the *unit * size* instead of direct notation with digits. Interestingly, all the default segments are divisible by 8, that leads bitstring values to be accepted as binary types.

Segment Type = integer | float | binary | bytes | bitstring | bits | utf8 | utf16 | utf32

Figure 8: Bitstring Segment Types

Despite the rare use cases, bitstring types are still designed to be recognize *unit* and *size* other than digits divisible by 8 and these are not strictly accepted as **binary()**. These types can be addressed as **binary()** and **dynamic()**. Since bitstrings exist to generalize binaries as mentioned above, the specification should describe a binary type and otherwise realized at compile time.

$$\begin{aligned}
\llbracket \langle \rangle \rrbracket &\approx \text{binary}() \text{ and } \text{dynamic}() \\
\llbracket \langle _ :: n \rangle \rrbracket &\approx \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{binary}() \text{ and } \text{dynamic}() & \text{otherwise} \end{cases} \\
\llbracket \langle _ :: _ * n \rangle \rrbracket &\approx \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{binary}() \text{ and } \text{dynamic}() & \text{otherwise} \end{cases} \\
\llbracket \langle _ :: n, _ :: _ * n \rangle \rrbracket &\approx \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{binary}() \text{ and } \text{dynamic}() & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 9: Approximation of bitstring Types

4.2 Approximation of Top Function Types

The top function type is union of all the functions with every arity to the top type as defined in Figure 10. Since the top function type already takes over all the other function types, it becomes not worthy to have all the other function types that the return type is a subtype of the top type. In the perspective of memory space, it is only costly to define and have all the function types because the maximum arity is 255.

$$\dots \rightarrow \text{any}() \equiv \text{none} \rightarrow \text{any}() \mid \dots \mid (\text{none}())_1, \dots, \text{none}()_{255} \rightarrow \text{any}()$$

Figure 10: Top Function Type

However, Elixir Types will not directly define **fun()** as Figure 10, due to the triviality of defining the list of function with all the possible arity. While Elixir Types restricts the top type to be only **fun()**, keeping the semantic approach to the top function, TypeSpec has allowed to specify the return type of the top type which have to be taken into account when translating it. Nevertheless, this leniency in expressiveness means nothing in practice, given that the Dialyzer does not differentiate the return type but only perceive any form of the top types as $\dots \rightarrow \text{any}()$. These finally lead approximation of the any top function types into **fun()** and **dynamic()**.

$$\llbracket (\dots \rightarrow T) \rrbracket \approx \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{fun}() \text{ and } \text{dynamic}() & \text{otherwise} \end{cases}$$

Figure 11: Approximation of Top Function Type

4.3 Approximation of Range Types

```
[[neg_integer]] = infy..-1
[[non_neg_integer]] = 0..infy
[[pos_integer]] = 1..infy
[[n]] = n..n
[[n..n']] = n..n'
```

Figure 12: Range Types

4.4 Approximation of Field Types

Approximating field types is more delicate than the others hence care must be taken. That is TypeSpec allows all the types to be specified as key types of field types. Whereas, Elixir Types' syntax has separate key types so that the type system does not deal with infinite set of types when it comes to field typing. There comes four cases when the field types are not key types:

- Type variables.
- Subtypes of key types.
- Supertypes of key types.
- Neither subtypes nor supertypes of key types.

The first case is simple since they are eventually types belong to the syntax. All it requires is to recursively evaluate it.

As most of the types are evaluated as subtypes of key types, it is sound to under-approximate them to their supertypes in key types. For example, Figure 13 shows a simple identity function that takes a record.

```
@spec record_val(%{{atom()} => integer()}) :: integer()
def record_val(x) when is_map_key(x, {:ok}), do: Map.get(x, {:ok})

[[@spec id_record(%{{atom()} => integer()}) :: integer()]]
    ⋈
[[@spec id_record(%{tuple() => integer()}) :: integer()]]
    ⋈
$ %{tuple() => integer()} -> integer()
```

Figure 13: Record Value Access Function

The key type is specified to accept tuple with a single atom type, but Dialyzer will pass when the example function is applied to records with any type of tuples as key types. That is complemented by conservative approximations of Elixir Types in pattern/guard analysis: it states that a type cannot express the exact domain of a function and henceforth under-approximation is used with the guards/pattern that can filter out unintended types of values, which is called potentially accepted type.

$$\begin{aligned}
\llbracket [] \rrbracket &\lesssim \text{list}() \\
\llbracket \{\bar{T}\} \rrbracket &\lesssim \text{tuple}() \\
\llbracket (\bar{T} \rightarrow T) \rrbracket &\lesssim \text{fun}()
\end{aligned}$$

Figure 14: Subtypes of Keytypes

`list()` in key types is semantically a set of all the list types and `nonempty_maybe_improper_list()` in TypeSpec is a basic type to describe all kind of lists including the empty list and improper lists. What hinders us from direct translation is that `list()` does not include improper lists hence to be a subtype of `nonempty_maybe_improper_list()`. Notwithstanding, this is not an interesting part for the improper lists not meant to be used as the name suggests. The over-approximation of `nonempty_maybe_improper_list()`, therefore, is sound to be `list()`.

$$\llbracket \text{nonempty_maybe_improper_list}(T_1, T_2) \rrbracket \gtrsim \text{list}()$$

Figure 15: Supertypes of Keytypes

Even though the last cases in TypeSpec are not in the key types in Elixir Types, they can be semantically translated. Firstly, since `|` is not a type but a type operator, it can simply be approximated per se. Following Figure 16 displays different syntaxes with the same semantics.

$$\begin{aligned}
&\llbracket @spec \text{fun}(\%{\text{atom}()} \mid \text{binary}() \Rightarrow \text{integer}()) :: \text{integer}() \rrbracket \\
&\quad \rightsquigarrow \\
&\quad \$ \%{\text{atom}() \text{ or } \text{binary}() \Rightarrow \text{integer}()} \rightarrow \text{integer}() \\
&\quad \rightsquigarrow \\
&\quad \$ \%{\text{atom}() \Rightarrow \text{integer}() \text{ or } \text{binary}() \Rightarrow \text{integer}()} \rightarrow \text{integer}() \\
&\quad \rightsquigarrow \\
&\quad \$ \%{\text{atom}() \Rightarrow \text{integer}()} \text{ or } \%{\text{binary}() \Rightarrow \text{integer}()} \rightarrow \text{integer}()
\end{aligned}$$

Figure 16: Semantical Comprehension of Or Operator

With a little autopsy on `any()` and `none()` as key types, comprehending it does not require a lot of effort; what they do in field types. `any()` as a key type expands to unions of all the types mapping to a value type. On the contrary, `none()` eradicates the field in the record type, for there is no field type without a key type. Imagine a type system only with a set of types, i.e., `{any(), none(), integer(), float(), binary()}`. Then, the translation of `%{any() => integer()}` expands it to `%{(none() or integer() or float() or binary()) => integer()}` Whereas, `%{none() => any()}` is collapsed into `%{}` which is an empty record type. It all seems easy and sound, except one nuisance that risen by the need of listing all the types to translate `any()` in key types.

$$\begin{aligned}
\llbracket \text{required}(T) \Rightarrow T' \rrbracket &\approx \begin{cases} [\text{ : } k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx \text{ : } k \\ [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \not\approx \text{ : } k \\ & \text{if } \llbracket T \rrbracket \not\approx \bigvee_{i \in I} K_i \\ & \text{if } \llbracket T \rrbracket \not\leq \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \approx T \setminus \bigvee_{i \in I} K_i \end{cases} \\
\llbracket \text{optional}(T) \Rightarrow T' \rrbracket &\approx \begin{cases} [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \\ \text{undefined} & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 17: Approximation of Field Types